

CSCE 676 DATA MINING · MODULE 02 · LECTURE NOTES

Similar Items at Scale

The field's model result: exact becomes approximate, with a bound.

MODULE OUTCOMES

- Represent documents as sets and justify the choice.
- Explain minhashing and why it estimates Jaccard.
- Explain LSH and derive its S-curve.
- Choose LSH parameters from a target threshold.
- Verify the approximation bounds empirically.

1 Sets and shingles

REPRESENT A DOCUMENT AS THE SET OF ITS K-CHARACTER SHINGLES

A **k-shingle** is any contiguous substring of length k — so a document becomes the set of all such substrings it contains, and **this preserves local word order** in a way that a bag of words does not, which matters because reordered text is not a near-duplicate.

And k matters a good deal: $k = 5$ suits short documents and $k = 9$ or 10 suits ordinary prose — **too small and every document shares shingles purely by chance, which destroys the signal; too large and genuine near-duplicates share none of them.** The rule of thumb is that the shingle space should be large enough that a typical document uses a tiny fraction of it.

Then hash the shingles to fixed-width integers, which bounds the space, makes the sets cheap to store, and makes intersection a numeric operation — at the cost of occasional collisions, which are harmless at the rates involved.

So the similarity question becomes a set similarity question — and **Jaccard similarity, the size of the intersection over the size of the union, is the measure that follows naturally**, because it is insensitive to document length.

$$J(A, B) = |A \cap B| / |A \cup B|$$

- Between 0 and 1, and insensitive to document length — which is why it suits near-duplicate detection, where one document may be a superset of another with boilerplate added.
- Computing it for all pairs requires $C(n, 2) \approx n^2/2$ comparisons. At $n = 10^6$ that is 5×10^{11} set intersections — infeasible, and that is only a million documents, which is small by this course's standards.

- **So the approach is: estimate J cheaply, and then avoid most pairs entirely — two separate ideas, which minhashing and LSH address respectively, and both are necessary:** a cheap estimator still leaves you with $n^2/2$ estimates to compute.
- **Confusing the two is the usual error**, and separating them is what makes this module tractable — so it is worth being able to state which problem each half solves.

2 Minhashing

THE PROBABILITY THAT TWO SETS HAVE THE SAME MINHASH IS EXACTLY THEIR JACCARD SIMILARITY

Take a random permutation of the universe of possible shingles, and record, for each set, the first of its own elements in that order. That single value is the set's minhash under that permutation.

And $P[\text{minhash}(A) = \text{minhash}(B)] = J(A, B)$, exactly — because the first element of $A \cup B$ under a uniformly random permutation is equally likely to be any element of the union, and the two minhashes agree precisely when that element lies in the intersection. The proof is one line and worth working through.

So averaging over many independent permutations estimates J — the fraction of positions at which two signatures agree is an unbiased estimator of their Jaccard similarity, and its variance falls as $1/k$ in the number of permutations.

Which gives the signature: replace a set of any size with k integers, typically 100 to 200, with the standard error around $1/\sqrt{k}$ — and that is CSCE 658's concentration machinery doing the work, which is why this course requires it.

THE PROBLEM WITH REAL PERMUTATIONS

a permutation of a billion shingles is a billion numbers, per permutation. Unaffordable, and you need hundreds of them.

THE FIX: use hash functions as pseudo-permutations

$h_i(x) = (a_i * x + b_i) \bmod p$
 with random a_i , b_i and p a large prime. Each h_i stands in for one permutation.

ONE PASS OVER THE DATA

```
for each set S:
  for each i in 1..k:
    sig[i] = min over x in S of h_i(x)
```

SO the signature is k minima, computed in a single pass, with no permutation ever materialised.

AND THE ERROR

the estimate is a mean of k Bernoulli trials, so the standard error is about $\sqrt{J(1-J)/k}$.
 $k = 200$ gives roughly 0.035 -- which Project 1 asks you to verify rather than assume.

Hash functions standing in for permutations is the implementation trick, and it is why the method is one pass and constant memory per set — which is exactly Module 01 §2's cost model being respected. **This is the detail students most often miss**, and it matters because the quality of the hash family determines whether the independence the bound assumes actually holds (§4's last point).

3 Locality-sensitive hashing

HASH SIGNATURES INTO BANDS SO THAT SIMILAR ITEMS COLLIDE AND DISSIMILAR ONES MOSTLY DO NOT

Split each k -element signature into b bands of r rows each, with $br = k$ — and hash each band separately into buckets, so each item lands in b buckets.

Two items become candidates if they land in the same bucket for *any* band — so you only compute the actual similarity for pairs that collided somewhere, which is a tiny fraction of all pairs and is where the quadratic cost disappears.

And the probability of becoming a candidate is a function of the true similarity: $1 - (1 - s^r)^b$ — the probability of agreeing on all r rows of at least one of b bands — which is an S-shaped curve in s .

Which means the threshold is tunable rather than fixed: the curve's steep region sits near $(1/b)^{1/r}$, so choosing b and r places the threshold wherever your application needs it — which is the design freedom the construction buys.

$$P(\text{candidate} \mid s) = 1 - (1 - s^r)^b \quad \text{threshold} \approx (1/b)^{1/r}$$

- Both parameters appear, and they do different things. Increasing r sharpens the curve (a steeper transition), and increasing b shifts it left (a lower threshold, more candidates).
- The threshold is the similarity at which the curve crosses the middle — pick this value from your application requirement, then solve for b and r subject to $br = k$.
- More bands means more candidates, higher recall, and more work — the trade is false positives against false negatives, and the construction lets you choose where on it to sit rather than accepting a default.
- Plot the curve before choosing parameters. The trade is visible on the plot and essentially invisible in the numbers — which is why Project 1 requires the plot rather than the parameters, and why tuning by trial misses the structure.

4 Beyond Jaccard

- **Cosine similarity: random hyperplane hashing.** Take the sign of the dot product with a random vector — and the collision probability is $1 - \theta/\pi$, where θ is the angle, which decreases with distance exactly as required.
- **Euclidean distance: random projection into buckets.** Project each point onto a random line and quantise the result, so that nearby points usually fall in the same bucket and distant ones usually do not.
- **Hamming distance: sample random bit positions.** The simplest family and the easiest to analyse — the collision probability is the fraction of agreeing bits raised to the number sampled.
- And the general requirement is always the same: a hash family in which the collision probability decreases with distance — which is what makes the banding amplification work, and is the definition of locality-sensitive.

- Which is how approximate nearest-neighbour search works, and is what CSCE 670 Module 08 builds on for dense retrieval at scale. The abstraction is worth holding: any distance with a locality-sensitive hash family gets the same banding construction and the same S-curve analysis, which means learning it once covers all of them.

AND WHAT TO VERIFY RATHER THAN ASSUME

Measure the minhash estimate against the exact Jaccard similarity on a sample — and compare the observed error to $\sqrt{(J(1-J)/k)}$, which is Project 1's central requirement and takes an hour.

Report both error rates. Recall alone hides the trade — a parameter setting with excellent recall and a huge candidate count has not saved you anything — and the candidate count is what determines whether the method actually helped.

And check the hash functions. A poor hash family breaks the independence the bound assumes, which shows up as correlated collisions rather than as an obvious error — so the estimates are biased in a way that looks like a property of the data.

Which is Module 01 §2's discipline in its first concrete instance: choose the approximation, state the bound, then measure whether it holds on your data. Every remaining module in this course has a version of this paragraph.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Mining of Massive Datasets, chapter 3 (free)	The whole module, and the best treatment in print — including the parameter selection worked through numerically.
Broder — On the resemblance and containment of documents (free)	§2 in the original — minhashing as it was invented, for exactly this problem.
Indyk & Motwani — Approximate nearest neighbors (free)	§4's general framework — where locality-sensitive hashing is defined properly.
Charikar — Similarity estimation from rounding algorithms (free)	§4's first row — the random hyperplane family and its collision probability.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Shingle a document at $k = 3, 5$, and 10 and compare the set sizes.

- 2 **Compute Jaccard similarity for ten document pairs** exactly.
- 3 **Extrapolate the all-pairs cost** to a million and a billion documents.
- 4 **Implement minhashing** with hash-based pseudo-permutations.
- 5 **Verify that the agreement fraction estimates Jaccard** on pairs whose true similarity you know.
- 6 **Plot the observed error against k** and compare to $1/\sqrt{k}$.
- 7 **Implement banded LSH** and plot the S-curve for three (b, r) choices.
- 8 **Derive b and r from a target threshold of 0.8** with $k = 200$.
- 9 **Report candidate count, false positives, and false negatives** at your chosen parameters.
- 10 **Implement random hyperplane hashing** for cosine similarity and verify its collision probability.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What is a shingle, and how do you choose k ?
- 2 Why does Jaccard suit this problem?
- 3 What are the two separate problems, and what solves each?
- 4 State the minhash property and sketch why it holds.
- 5 Why are hash functions used instead of permutations?
- 6 Give the standard error of a k -permutation estimate.
- 7 Describe the banding construction and the candidate condition.
- 8 Give the S-curve formula and the approximate threshold.
- 9 What do b and r each control?
- 10 Name three other LSH families and the general requirement.